Last updated on Friday, 13 March 2026

CAB320 Assignment 1 - Sokoban-game

Intelligent Search – Motion Planning in a Warehouse

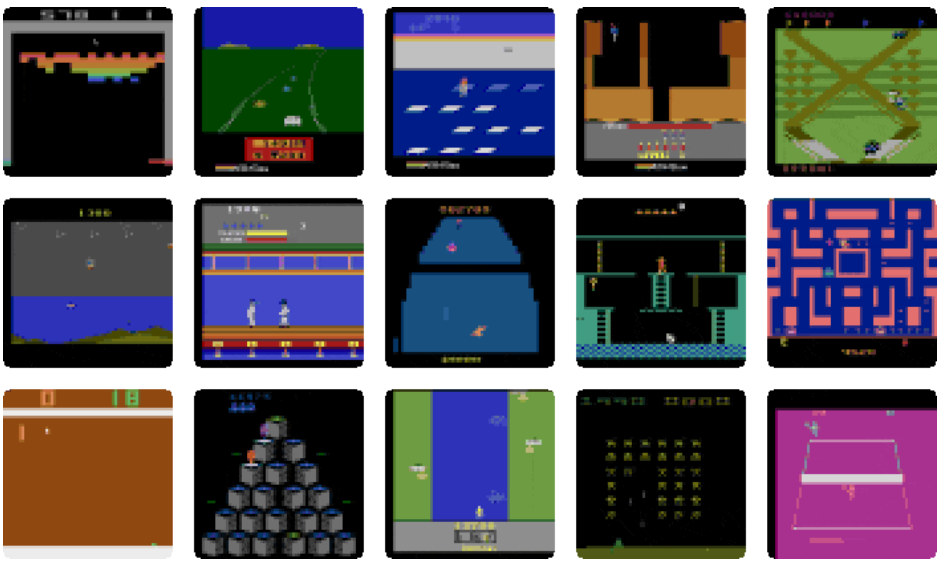
Key information

- Submission due date is available on Canvas
- Submit your work via Canvas
- Group size: 3 people (max) per submission (2 people is acceptable if preferred, but completion of the same tasks is required.). No group of 1 (this is not a 'group').

Overview

Artificial Intelligence researchers enjoy playing computer games [not a surprise!]. From checkers to chess to Star Craft and DOTA, much effort has been devoted to creating intelligent agents to address these problems. Apart from the fun of playing games and seeing if AI can do as well or better than humans, there are number of reasons why developing AI for games is worthwhile:

1. AI can create interesting adversaries for humans; from chess computers to help people learn and improve their games to NPCs (Non-Player Characters, where occasionally the AI has to be "dumbed down" to make them plausible to beat).
2. Demonstrating the level of AI advancement, e.g. AlphaGo main international headlines by beating a top-level Go player, which then lead to a noble prize for protein folding on the follow-up AlphaFold system [https://deepmind.google/technologies/alphafold/].
3. Games are 'toy' problems in that they have well-defined characteristics (such as the established rules of Chess), which allow specific characteristics of an AI algorithm to be investigated and improved (e.g. search strategy for looking ahead a number of turns).



https://www.verses.ai/research-blog/achieving-human-level-atari-gameplay-with-bayesian-object-priors-and-active-inference

Figure 1. Atari games used to develop AI

Many approaches *toy* problems are not trivial in terms of requiring large computation power, sophisticated algorithms and months of AI development by teams of engineers. For this assignment we have chosen an interesting game that is soluble on standard available computation, can be run

quickly and illustrates important AI concepts.

Sokoban is a computer puzzle game in which the player pushes boxes around a maze in order to place them in designated locations. It was originally published in 1982 for the Commodore 64 and IBM-PC and has since been implemented in numerous computer platforms and video game consoles.

It illustrates AI for sequential problem solving with soft and hard constraints, which has real-life applications - such as warehouse logistics in package distribution warehouses. The screenshot below shows the GUI provided for the assignment. It models a robot moving boxes in a warehouse and as such, it can be treated as an automated planning problem. Sokoban is an interesting challenge for the field of artificial intelligence largely due to its difficulty. Sokoban has been proven NP-hard (small scale problems are tractable, but as the environment grows then searching all possible solutions becomes impracticable).

Sokoban is difficult not because of its branching factor, but because of the huge depth of the solutions. Many actions (box pushes) are needed to reach the goal state! However, given that the only available actions are moving the worker up, down, left or right, the branching factor is small (only 4).

The worker can only push a single box at a time and is unable to pull any box. The boxes have individual weights. The weight of a box is taken into account when computing the cost of a push. The cost of an action is 1 + weight of the box being pushed (if any). That is, if the worker moves to an empty cell the action cost is 1. If the worker pushes a box that has a weight of 7, the action cost is 8 = 1+7.

The aim of this assignment is to design and implement a planning agent for Sokoban

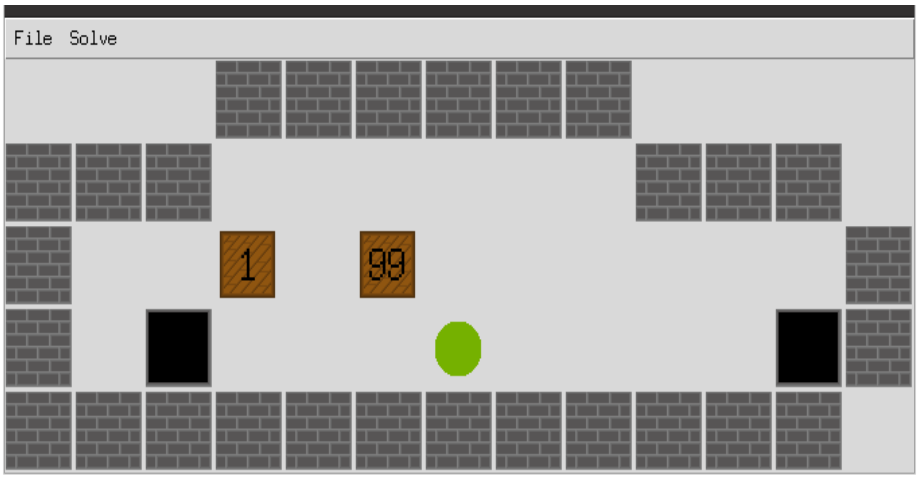


Illustration 1: Initial state of a warehouse. The green disk represents the agent/robot/player, the brown squares represent the boxes/crates. The black cells denote the target positions for the boxes. The left box has a weight of 1. The right box has a weight of 99.

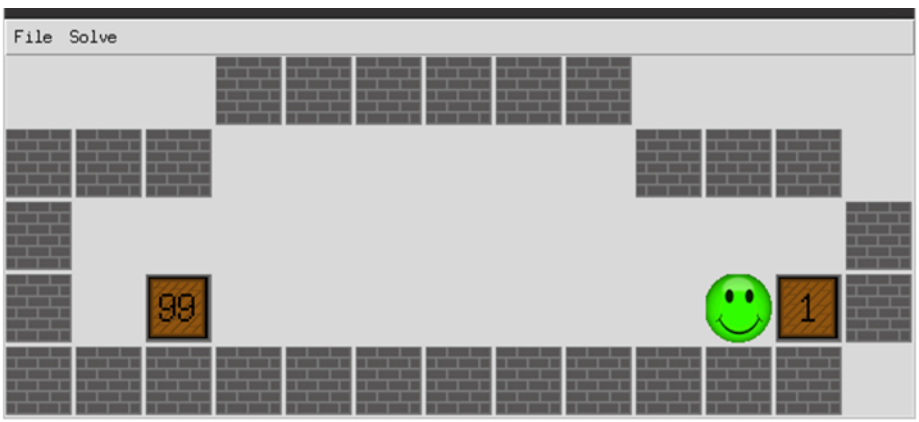


Illustration 2: Goal state reached: all the boxes have been pushed to a target position. Notice that the light box (weight of 1) has been moved to the target that was initially the farthest.

Approach

As already mentioned, Sokoban has a large search space with few goals, located deep in the search tree. However, admissible heuristics can be easily obtained. These observations suggest adopting an informed search approach. Suitable generic algorithms include A* and its variations.

You will implement a weighted variant of the classical Sokoban. In this variant, we assign an individual pushing cost to each box, whereas for the classical Sokoban, we simply count the number of actions executed.

In order to help you create an effective solver, you are given a few python scripts and a template (see the python files provided in ***CAB320-A1-Student-Template.zip***, for further details). You are asked to implement a few auxiliary functions, within the ***mySokobanSolver.py*** file (NB: This is the only python file that needs changing).

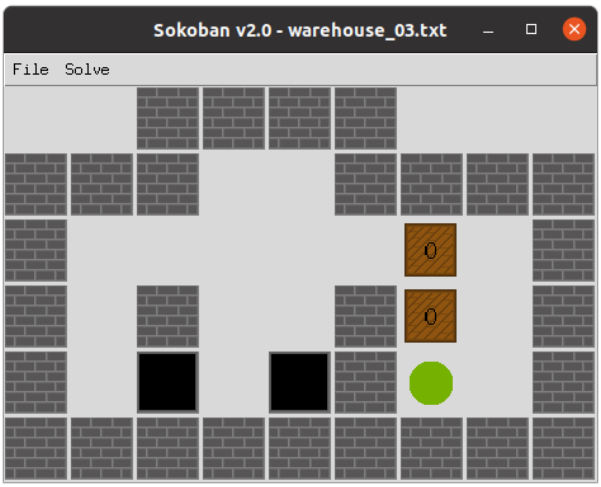
Warehouse representation in text files

To help you design and test your solver, you are provided with a number of puzzles. The first line of the file is the weights of the boxes indexed in the order the boxes are encountered when you scan the warehouse row by row. If the text file contains no weights, they are assumed to be 0.

The puzzles and their initial state are coded as follows in the text files,

- **space**, a free square
- '#', a wall square
- '**$**', a box
- '**.**', a target square
- '@', the player
- '**!**', the player on a target square
- '*', a box on a target square

For example, the puzzle below is coded in a text file as




```
    #   #   #   #
#   #   #       #   #   #   #
#               #   $       #
#       #       #   $       #
#       .   .   #   @       #
#   #   #   #   #   #   #   #
```

Files provided

- **search.py** contains a number of search algorithms and related classes.
- **sokoban.py** contains a class ***Warehouse*** that allows you to load puzzle instances from text files.
- **gui_sokoban.py** a GUI implementation of Sokoban that allows you to play and explore puzzles. This GUI program can call your planner function *solve_weighted_sokoban*
- **mySokobanSolver.py** code skeleton for your solution. You should complete all the functions located in this file. This is the only python file that you should submit.
- **sanity_check.py** script to perform very basic tests on your solution. The marker will use a different script with different warehouses. You should develop your own tests to validate your code, i.e. create unit-tests, simple tests, complex tests and 'wicked' tests.
- A number of puzzles in the folder 'warehouses'.

Here is a description of how the taboo cells work:

A "taboo cell" is by definition

- a cell inside a warehouse such that whenever a box get pushed on such
- a cell then the puzzle becomes unsolvable.

Cells outside the warehouse are not taboo. It is a fail to tag one as taboo.

When determining the taboo cells, you must ignore all the existing boxes, only consider the walls and the target cells.

Use only the following rules to determine the taboo cells;

- Rule 1: if a cell is a corner and not a target, then it is a taboo cell.
- Rule 2: all the cells between two corners along a wall are taboo if none of these cells is a target.

Your tasks

Your solution **has to comply** with the assignment framework for consistency in code style (this helps the tutors help you if you have any questions). That is, you have to use the classes and functions provided in the file *search.py.*

[Much of the functionality has been introduced in the practicals, so should be familiar]

All your code should be located in a single file called ***mySokobanSolver.py***. **This is the only Python file that you should submit to gradescope, compressed within a zip folder**. In this file, you will find partially completed functions and their specifications. You can add auxiliary classes and functions to this file. When your submission is tested, it will be run in a directory containing the files *search.py* and *sokoban.py* and your file *mySokobanSolver.py*. **If you break this interface, your code will fail the tests!**

Deliverables

You should submit via Canvas only two files

1. An **individual report** in **pdf** format **strictly limited to 4 pages in total** (be concise!):
    - One section that explains clearly your state representation, your heuristics, and any other important features needed to understand your solver.
    - Once section on your testing methodology. How did you validate your code? Did you create toy problems to verify that your code behaves as expected?
    - Once section that describes the performance and limitations of your solver.
2. Your (group) **Python scripts in** the one file: **mySokobanSolver.py, zipped for submission to Gradescope.**

Note we will be using Gradescope to mark the code for this assignment. To submit your code to Gradescope, you must compress your code into a zip archive containing only the file mySokobanSolver.py and submit it to Assignment 1: Planning in a Warehouse in the Gradescope tab on Canvas.

Once you have submitted your code, you will receive some immediate feedback about whether the four marked functions in your code behave as expected and that your code runs. This can be used to verify the output type is correct but does NOT guarantee the output itself and your code is correct (e.g. you will get some feedback that taboo_cells outputs the correct data type (string) but not that the string is correct). You may use Gradescope for testing code functionality throughout the assignment but your most recent submission at the due date will be used to determine your grade.

Marking Guide Overview

- **Report**: 10 marks
    - Structure (sections, page numbers), grammar, no typos.
    - Clarity of explanations.
    - Figures and tables (use for explanations and to report performance).
- **Code quality**: no marks but the following helps you debug!
    - Readability, meaningful variable names.
    - Proper use of Python idioms like dictionaries and list comprehension.
    - Header comments in classes and functions. In-line comments.
    - Function parameter documentation.
- **Functions of mySokobanSolver.py : 20 marks** (from 30 points)

The markers will run python scripts to test your functions. Make sure that you respect the specifications of these functions. Otherwise, your functions will fail the tests, and you will get low marks for functionality!

- **my_team():** 1 point
- **taboo_cells**(): 5 points
- **check_elem_action_seq()**: 4 points
- **solve_weighted_sokoban()**: 20 points

The total of points is then rescaled to out 20 marks (for the 20% weight of the group component).

Recommendations

- Do not underestimate the workload. Start early. You are strongly encouraged to ask questions during the practical sessions or use the assignment channel on Teams.
- Don't forget to **list all the members of your group in the report and the code**!
- Each person in your group should submit the assignment.
- Enjoy the assignment!

Frequently Asked Questions (FAQ)

Running time; Here are examples of running time of a model solution (no optimization!)

- warehouse_07.txt is medium difficulty,
  Analysis took 300.645556 seconds
  Solution found with a cost of 26
  ['Up', 'Up', 'Right', 'Right', 'Up', 'Up', 'Left', 'Left', 'Down', 'Down', 'Right', 'Up', 'Down', 'Right', 'Down', 'Down', 'Left', 'Up', 'Down', 'Left', 'Left', 'Up', 'Left', 'Up', 'Up', 'Right']
- warehouse_09.txt is easy
  Analysis took 0.009575 seconds
  Solution found with a cost of 396
  ['Up', 'Right', 'Right', 'Down', 'Up', 'Left', 'Left', 'Down', 'Right', 'Down', 'Right', 'Left', 'Up', 'Up', 'Right', 'Down', 'Right', 'Down', 'Down', 'Left', 'Up', 'Right', 'Up', 'Left', 'Down', 'Left', 'Up', 'Right', 'Up', 'Up', 'Left', 'Left', 'Down', 'Left', 'Left', 'Down', 'Down', 'Right', 'Right', 'Right', 'Right', 'Right', 'Right', 'Down', 'Right', 'Right', 'Up', 'Left', 'Left', 'Left', 'Left', 'Left', 'Down', 'Left', 'Left', 'Up', 'Up', 'Up', 'Right', 'Right', 'Right', 'Up', 'Right', 'Down', 'Down', 'Up', 'Left', 'Left', 'Left', 'Left', 'Down', 'Down', 'Down', 'Right', 'Right', 'Up', 'Right', 'Right', 'Left', 'Left', 'Down', 'Left', 'Left', 'Up', 'Right', 'Right']
- warehouse_47.txt is easy
  Analysis took 0.122561 seconds
  Solution found with a cost of 179
  ['Right', 'Right', 'Right', 'Up', 'Up', 'Up', 'Left', 'Left', 'Down', 'Right', 'Right', 'Down', 'Down', 'Left', 'Left', 'Left', 'Left', 'Up', 'Up', 'Right', 'Right', 'Up', 'Right', 'Right', 'Right', 'Right', 'Down', 'Left', 'Up', 'Left', 'Down', 'Down', 'Up', 'Up', 'Left', 'Left', 'Down', 'Left', 'Left', 'Down', 'Down', 'Right', 'Right', 'Right', 'Right', 'Right', 'Right', 'Down', 'Right', 'Right', 'Up', 'Left', 'Left', 'Left', 'Left', 'Left', 'Down', 'Left', 'Left', 'Up', 'Up', 'Up', 'Right', 'Right', 'Right', 'Up', 'Right', 'Down', 'Down', 'Up', 'Left', 'Left', 'Left', 'Left', 'Down', 'Down', 'Down', 'Right', 'Right', 'Up', 'Right', 'Right', 'Left', 'Left', 'Down', 'Left', 'Left', 'Up', 'Right', 'Right']
- warehouse_81.txt is easy
  Analysis took 0.170240 seconds
  Solution found with a cost of 376
  ['Left', 'Up', 'Up', 'Up', 'Right', 'Right', 'Down', 'Left', 'Down', 'Left', 'Down', 'Down', 'Down', 'Right', 'Right', 'Up', 'Left', 'Down', 'Left', 'Up', 'Right', 'Up', 'Up', 'Left', 'Left', 'Down', 'Right', 'Up', 'Right', 'Up', 'Right', 'Up', 'Up', 'Left', 'Left', 'Down', 'Down', 'Right', 'Down', 'Down', 'Left', 'Down', 'Down', 'Right', 'Up', 'Up', 'Up', 'Down', 'Left', 'Left', 'Up', 'Right']
- warehouse_147.txt is medium difficulty
  Analysis took 197.910769 seconds
  Solution found with a cost of 521
  ['Left', 'Left', 'Left', 'Left', 'Left', 'Left', 'Down', 'Down', 'Down', 'Right', 'Right', 'Up', 'Right', 'Down', 'Right', 'Down', 'Down', 'Left', 'Down', 'Left', 'Left', 'Up', 'Up', 'Down', 'Down', 'Right', 'Right', 'Up', 'Right', 'Up', 'Up', 'Left', 'Left', 'Left', 'Down', 'Left', 'Up', 'Up', 'Up', 'Left', 'Up', 'Right', 'Right', 'Right', 'Right', 'Right', 'Right', 'Down', 'Right', 'Right', 'Right', 'Up', 'Up', 'Left', 'Left', 'Down', 'Left', 'Left', 'Left', 'Left', 'Left', 'Left', 'Down', 'Down', 'Down', 'Right', 'Right', 'Up', 'Left', 'Down', 'Left', 'Up', 'Up', 'Left', 'Up', 'Right', 'Right', 'Right', 'Right', 'Right', 'Right', 'Left', 'Left', 'Left', 'Left', 'Left', 'Down', 'Down', 'Down', 'Down', 'Right', 'Down', 'Down', 'Right', 'Right', 'Up', 'Up', 'Right', 'Up', 'Left', 'Left', 'Left', 'Down', 'Left', 'Up', 'Up', 'Up', 'Left', 'Up', 'Right', 'Right', 'Right', 'Right', 'Right', 'Down', 'Right', 'Down', 'Right', 'Right', 'Up', 'Left', 'Right', 'Right', 'Up', 'Up', 'Left', 'Left', 'Down', 'Left', 'Left', 'Left', 'Left', 'Left', 'Left', 'Right', 'Right', 'Right', 'Right', 'Right', 'Right', 'Up', 'Right', 'Right', 'Down', 'Down', 'Left', 'Down', 'Left', 'Left', 'Up', 'Right', 'Right', 'Down', 'Right', 'Up', 'Left', 'Left', 'Up', 'Left', 'Left']
- warehouse_111.txt is hard; did not complete overnight
- warehouse_5n is impossible
  Analysis took 1.504564 seconds
  Solution found with a cost of None
  Impossible

State representation; is a Warehouse a good state representation? Here are some clues.

- Think about what is static and what is dynamic in the problem.
- Where do you think static things should go? Problem instance or state?
- Where do you think dynamic things should go? Problem instance or state?

Where should we start the assignment?

- First, make sure that you understand the Problem classes that we saw in the pracs (the sliding puzzle and the pancake puzzle). Then implement in the following order the functions
    1. taboo_cells
    2. check_elem_action_seq
    3. solve_weighted_sokoban

Computation time and marking (CRA)

- With respect to the computation time, we will test the submissions on warehouses that require at most a couple of seconds with our solution.
- We will let your submissions run for one minute before aborting them. If they return within one minute, then the submission is considered acceptable with respect to time.
- The markers will run a test set on each submitted function.
- The test marks are binary: either correct or incorrect. If we run 10 tests for a function and you get 8 correct, your mark for this function will be 8/10.
- For functions like solve_*weighted_sokoban*, your returned solution does not have to be the same as our solution, but it needs to be of the same minimal cost to be considered correct.

How many submissions can I make?

- You can make multiple submissions. Only the last one submitted before the deadline will be marked.

How do I find teammates?

- Use the Canvas Discussions channel 'Group Searching (for Assignments)'
- Self-enrol in one of the Assignment 1 – Planning groups on the People page in Canvas (NB: every student needs to personally self-enrol in the group of your choosing, just agree on the number you are choosing as a team).
- Make sure you discuss early workload with your team-mates. It is not uncommon to see groups starting late or not communicating regularly and eventually submitting separate bits of work.
- Agree on the main form of communication you will be using in your group, and make sure you have a back up.
- Discuss and agree on expectations and tasks/responsibilities breakdown.
- If one of your teammates becomes unresponsive, put deadlines and submit without them if appropriate.